ITF23019: Parallel and Distributed Programming

Report for Lab 10: Message Passing Interface in Java

Name: Emil Berglund

GitHub: EmilB04

Disclaimer:

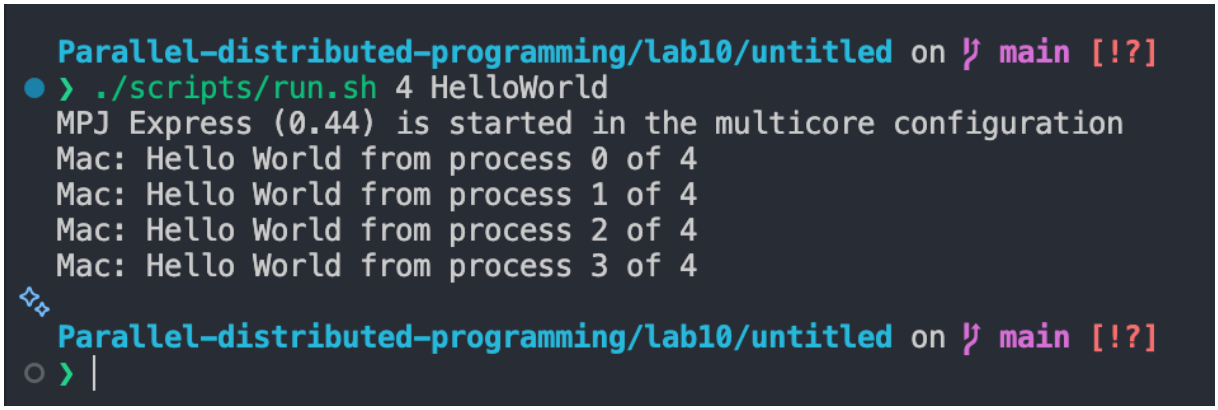
I used Visual Studio Code on Mac to solve this lab, and I could therefore not follow the provided guides in the PDF. The way I did it was asking Claude to generate some scripts to make this work for me, with instructions.

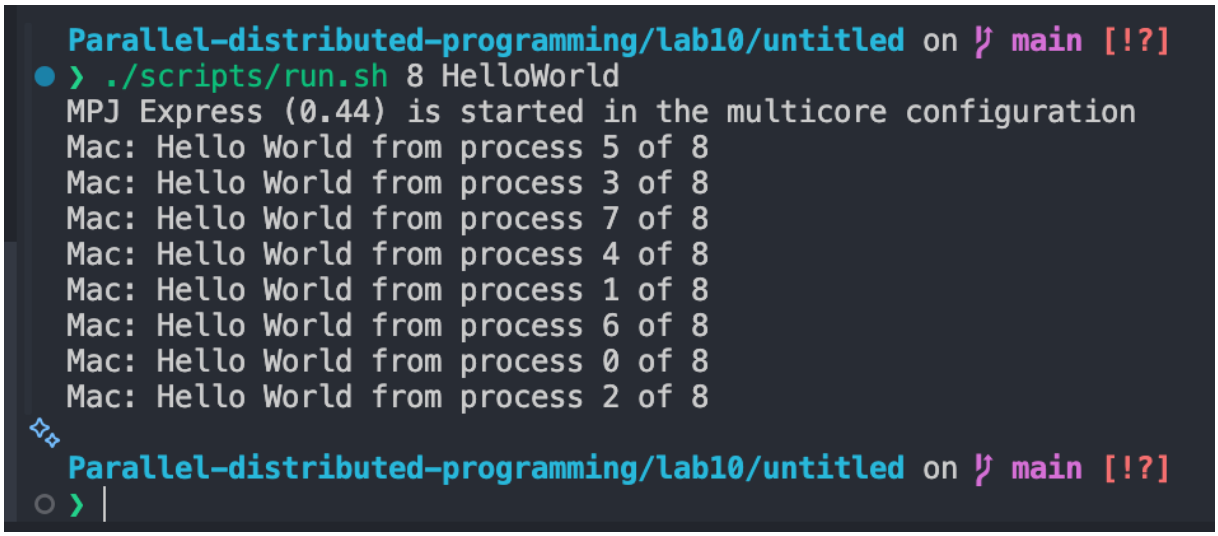
I will therefore commit a folder called *scripts* which includes some setup, running and compiling- scripts for running the provided code.

If interested, a README is provided at root level in *untitled*, explaining how to use the scripts.

Exercise 1: Hello.java

Subtask 1: Run code multiple times with different processes and include screenshots





```
Parallel-distributed-programming/lab10/untitled on  main [!?]
❯ ./scripts/run.sh 12 HelloWorld
MPJ Express (0.44) is started in the multicore configuration
Mac: Hello World from process 8 of 12
Mac: Hello World from process 7 of 12
Mac: Hello World from process 6 of 12
Mac: Hello World from process 11 of 12
Mac: Hello World from process 0 of 12
Mac: Hello World from process 5 of 12
Mac: Hello World from process 4 of 12
Mac: Hello World from process 1 of 12
Mac: Hello World from process 2 of 12
Mac: Hello World from process 10 of 12
Mac: Hello World from process 9 of 12
Mac: Hello World from process 3 of 12

Parallel-distributed-programming/lab10/untitled on  main [!?]
❯
```

Subtask 2: Multicore or cluster?

- The program is currently running in multicore configuration, as shown in the output in subtask 1.
- This is MPJ Express's default mode when you don't specify a device. It simulates MPI on a single machine by launching multiple local processes.

Exercise 2: Echo.java

Subtask 1: Run code and include screenshots

```
Parallel-distributed-programming/lab10/untitled on  main [!?]
❯ ./scripts/run.sh 2 Echo
MPJ Express (0.44) is started in the multicore configuration
Process 0 sends number 95 to Process 1
Process 1 receives number 95 from Process 0
Process 1 receives number 54 from Process 0
Process 0 sends number 54 to Process 1
Process 0 sends number 34 to Process 1
Process 1 receives number 34 from Process 0
Process 1 receives number 77 from Process 0
Process 0 sends number 77 to Process 1
Process 0 sends number 3 to Process 1
Process 1 receives number 3 from Process 0
Process 0 sends number 89 to Process 1
Process 1 receives number 89 from Process 0
Process 0 sends number 85 to Process 1
Process 1 receives number 85 from Process 0
Process 1 receives number 26 from Process 0
Process 0 sends number 26 to Process 1
Process 0 sends number 1 to Process 1
Process 1 receives number 1 from Process 0
Process 1 receives number 19 from Process 0
Process 0 sends number 19 to Process 1
Process 0 sends number 15 to Process 1
Process 1 receives number 15 from Process 0
Process 1 receives number 6 from Process 0
Process 0 sends number 6 to Process 1
Process 0 sends number 53 to Process 1
Process 1 receives number 53 from Process 0
Process 0 sends number 35 to Process 1
Process 1 receives number 35 from Process 0
Process 1 receives number 92 from Process 0
Process 0 sends number 92 to Process 1
Process 0 sends number 54 to Process 1
Process 1 receives number 54 from Process 0
Process 1 receives number 89 from Process 0
Process 0 sends number 89 to Process 1
Process 0 sends number 64 to Process 1
Process 1 receives number 64 from Process 0
Process 0 sends number 96 to Process 1
Process 1 receives number 96 from Process 0
Process 1 receives number 90 from Process 0
Process 0 sends number 90 to Process 1
Process 0 sends number 71 to Process 1
Process 1 receives number 71 from Process 0

Parallel-distributed-programming/lab10/untitled on  main [!?]
❯
```

-

Subtask 2: Modify and run program, and include screenshots

```
Parallel-distributed-programming/lab10/untitled on  main [!?]
❯ ./scripts/run.sh 2 Echo
MPJ Express (0.44) is started in the multicore configuration
=== Process 0: Sending 5 messages ===
=== Process 1: Waiting to receive 5 messages ===
  Message 1: Sending number 95 to Process 1
  Message 1: Received number 95 from Process 0
  Message 2: Sending number 54 to Process 1
  Message 2: Received number 54 from Process 0
  Message 3: Received number 34 from Process 0
  Message 3: Sending number 34 to Process 1
  Message 4: Sending number 77 to Process 1
  Message 4: Received number 77 from Process 0
  Message 5: Sending number 3 to Process 1
  Message 5: Received number 3 from Process 0

Parallel-distributed-programming/lab10/untitled on  main [!?]
❯
```

- **Before**: The program sent 21 messages in a back-and-forth pattern (Process 0 sends, then receives in the same loop).
- **After**:
    - Send 5 clearly numbered messages from Process 0 to Process 1
    - Process 1 receives and prints each message sequentially
    - Added section headers to show what each process is doing
    - Added message numbering for clarity

Exercise 3: MatrixMatrixMult.java

Subtask 1: Analyze and explain code

- Process 0 starts by creating and filling matrices A and B with random numbers. It then shares the data with all other processes. Matrix B is sent in full to everyone, since every process needs it to compute its part. Matrix A is split by rows, and each process only receives the rows it is responsible for.
- Each process then independently multiplies its assigned rows of A against the full matrix B, producing the corresponding rows of the result matrix C. All processes do this at the same time, which is what makes it faster than a single-process solution.
- Once every process is done, they all send their computed rows back to process 0, which assembles them into the final matrix C. Process 0 also handles timing and printing the result.

Subtask 2: Run code with $N = 1500$ and processes is 1, 2,… 10 – Screenshot

- Below are copies of the results.

```
====== np=1 ======
MPJ Express (0.44) is started in the multicore configuration
Matrix-Matrix multiplication
Square matrix size: 1500x1500
Number of processes: 1
Start Time = 1774267897,000000
End Time = 1774267900,000000
Running Time = 3,000000
====== np=2 ======
MPJ Express (0.44) is started in the multicore configuration
Matrix-Matrix multiplication
Square matrix size: 1500x1500
Number of processes: 2
Start Time = 1774267900,000000
End Time = 1774267901,000000
Running Time = 1,000000
====== np=3 ======
MPJ Express (0.44) is started in the multicore configuration
Matrix-Matrix multiplication
Square matrix size: 1500x1500
```

```
Number of processes: 3
Start Time = 1774267901,000000
End Time = 1774267902,000000
Running Time = 1,000000
====== np=4 ======
MPJ Express (0.44) is started in the multicore configuration
Matrix-Matrix multiplication
Square matrix size: 1500x1500
Number of processes: 4
Start Time = 1774267902,000000
End Time = 1774267904,000000
Running Time = 2,000000
====== np=5 ======
MPJ Express (0.44) is started in the multicore configuration
Matrix-Matrix multiplication
Square matrix size: 1500x1500
Number of processes: 5
Start Time = 1774267904,000000
End Time = 1774267905,000000
Running Time = 1,000000
====== np=6 ======
MPJ Express (0.44) is started in the multicore configuration
Matrix-Matrix multiplication
Square matrix size: 1500x1500
Number of processes: 6
Start Time = 1774267905,000000
End Time = 1774267906,000000
Running Time = 1,000000
====== np=7 ======
MPJ Express (0.44) is started in the multicore configuration
Matrix-Matrix multiplication
Square matrix size: 1500x1500
Number of processes: 7
Start Time = 1774267906,000000
End Time = 1774267907,000000
Running Time = 1,000000
====== np=8 ======
MPJ Express (0.44) is started in the multicore configuration
Matrix-Matrix multiplication
Square matrix size: 1500x1500
Number of processes: 8
Start Time = 1774267907,000000
End Time = 1774267908,000000
Running Time = 1,000000
====== np=9 ======
MPJ Express (0.44) is started in the multicore configuration
Matrix-Matrix multiplication
Square matrix size: 1500x1500
Number of processes: 9
Start Time = 1774267908,000000
End Time = 1774267909,000000
```

```
Running Time = 1,000000
====== np=10 ======
MPJ Express (0.44) is started in the multicore configuration
Matrix-Matrix multiplication
Square matrix size: 1500x1500
Number of processes: 10
Start Time = 1774267909,000000
End Time = 1774267910,000000
Running Time = 1,000000
```

Subtask 3: Calculate speedup and discuss its value with the above setting

- Under is a copy

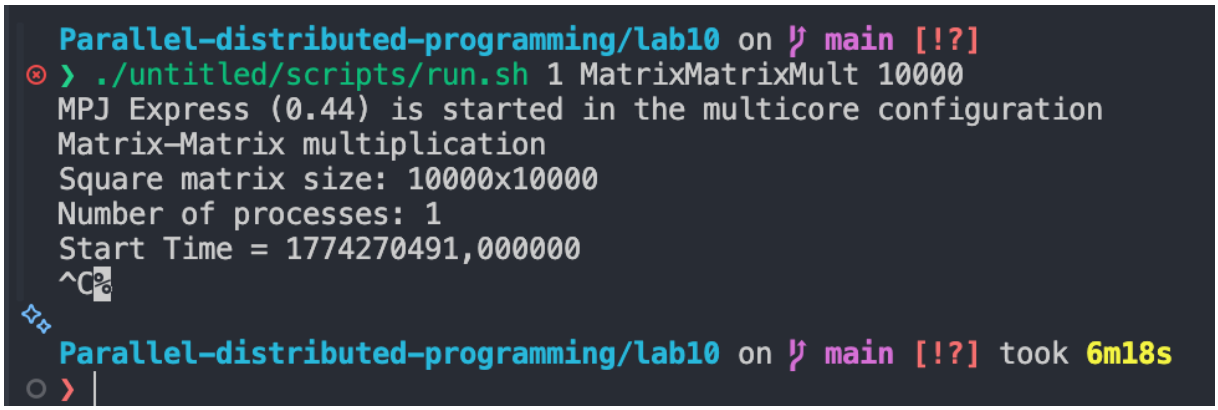
1. $p = 1$: $T_p = 3.0$, $S_p = 1.00$
2. $p = 2$: $T_p = 1.0$, $S_p = 3.00$
3. $p = 3$: $T_p = 1.0$, $S_p = 3.00$
4. $p = 4$: $T_p = 2.0$, $S_p = 1.50$
5. $p = 5$: $T_p = 1.0$, $S_p = 3.00$
6. $p = 6$: $T_p = 1.0$, $S_p = 3.00$
7. $p = 7$: $T_p = 1.0$, $S_p = 3.00$
8. $p = 8$: $T_p = 1.0$, $S_p = 3.00$
9. $p = 9$: $T_p = 1.0$, $S_p = 3.00$
10. $p = 10$: $T_p = 1.0$, $S_p = 3.00$

- For this setup ($N = 1500$, $p = 1 \ldots 10$), the measured speedup suggests performance improves quickly from one process but then mostly levels off around $S_p \approx 3$, with one outlier at $p = 4$ ($S_4 = 1.5$).
- These values should be interpreted carefully, because the runtime output is in whole-second steps (1s, 2s, 3s), which makes the results too unprecise for precise scaling analysis and can even produce unrealistic superlinear-looking values (for example $S_2 = 3$). So the current data is useful as a rough indication of parallel benefit, but not as a reliable measure of efficiency across process counts.

Subtask 4: What happens when $N = 10000$ – Explain.

For *N=10000*, the program needs a lot of memory and time. Each matrix is very large (about 800 MB), so the program may run out of memory. If it does run, it will still be very slow because there is much more calculation and data transfer between processes.

In my case I didn't get a end-time when waiting for over 6 minutes.



Exercise 4:

Subtask 1: Other MPI standards? Why or why not?

In principle all three programs in this lab can run on other Java MPI implementations, not only MPJ Express, because all three examples use common MPI concepts and mostly standard Java-MPI style calls.

Across HelloWorld.java, Echo.java, and MatrixMatrixMult.java, the code relies on the same core pattern:

1. Initialize/finalize MPI.
2. Get rank and size.
3. Use basic point-to-point operations (Send/Recv in Echo).
4. Use collective operations (Barrier, Bcast, Scatterv, Gatherv in matrix multiply).

This makes the source code broadly portable to mpiJava, FastMPJ, or similar libraries if they expose compatible mpi package APIs and collective support.

Subtask 2: How MPI APIs are similar and different in different MPI implementations

- Java MPI implementations are similar in the core model: Init/Finalize, Rank/Size, Send/Recv, and collectives like broadcast/gather.
- They differ in API details (method signatures, buffer/object handling, support for advanced collectives like Scatterv/Gatherv), runtime setup (launcher and environment), and performance/scalability.
- So MPI program logic is usually portable, but code adjustments and retesting are often needed when switching implementation.